

顺序关系符合要求 1, 但是函数得到的结果显然与我们定义的 `==` 不一致, 因此它不满足要求 2。

对于 `Sales_data` 的 `==` 运算符来说, 如果两笔交易的 `revenue` 和 `units_sold` 成员不同, 那么即使它们的 `ISBN` 相同也无济于事, 它们仍然是不相等的。如果我们定义的 `<` 运算符仅仅比较 `ISBN` 成员, 那么将发生这样的情况: 两个 `ISBN` 相同但 `revenue` 和 `units_sold` 不同的对象经比较是不相等的, 但是其中的任何一个都不比另一个小。然而实际情况是, 如果我们有两个对象并且哪个都不比另一个小, 则从道理上来讲这两个对象应该是相等的。

563

基于上述分析我们也许会认为, 只要让 `operator<` 依次比较每个数据元素就能解决问题了, 比方说让 `operator<` 先比较 `isbn`, 相等的话继续比较 `units_sold`, 还相等再继续比较 `revenue`。

然而, 这样的排序没有任何必要。根据将来使用 `Sales_data` 类的实际需要, 我们可能会希望先比较 `units_sold`, 也可能希望先比较 `revenue`。有的时候, 我们希望 `units_sold` 少的对象“小于”`units_sold` 多的对象; 另一些时候, 则可能希望 `revenue` 少的对象“小于”`revenue` 多的对象。

因此对于 `Sales_data` 类来说, 不存在一种逻辑可靠的 `<` 定义, 这个类不定义 `<` 运算符也许更好。

Best Practices

如果存在唯一一种逻辑可靠的 `<` 定义, 则应该考虑为这个类定义 `<` 运算符。如果类同时还包含 `==`, 则当且仅当 `<` 的定义和 `==` 产生的结果一致时才定义 `<` 运算符。

14.3.2 节练习

练习 14.18: 为你的 `StrBlob` 类、`StrBlobPtr` 类、`StrVec` 类和 `String` 类定义关系运算符。

练习 14.19: 你在 7.5.1 节的练习 7.40 (第 261 页) 中曾经选择并编写了一个类, 你认为它应该含有关系运算符吗? 如果是, 请实现它; 如果不是, 解释原因。

14.4 赋值运算符

之前已经介绍过拷贝赋值和移动赋值运算符 (参见 13.1.2 节, 第 443 页和 13.6.2 节, 第 474 页), 它们可以把类的一个对象赋值给该类的另一个对象。此外, 类还可以定义其他赋值运算符以使用别的类型作为右侧运算对象。

举个例子, 在拷贝赋值和移动赋值运算符之外, 标准库 `vector` 类还定义了第三种赋值运算符, 该运算符接受花括号内的元素列表作为参数 (参见 9.2.5 节, 第 302 页)。我们能以如下的形式使用该运算符:

```
vector<string> v;  
v = {"a", "an", "the"};
```

同样, 也可以把这个运算符添加到 `StrVec` 类中 (参见 13.5 节, 第 465 页):

```
class StrVec {  
public:  
    StrVec &operator=(std::initializer_list<std::string>);  
    // 其他成员与 13.5 节 (第 465 页) 一致  
};
```

564 为了与内置类型的赋值运算符保持一致（也与我们已经定义的拷贝赋值和移动赋值运算一致），这个新的赋值运算符将返回其左侧运算对象的引用：

```
StrVec &StrVec::operator=(initializer_list<string> il)
{
    // alloc_n_copy 分配内存空间并从给定范围内拷贝元素
    auto data = alloc_n_copy(il.begin(), il.end());
    free(); // 销毁对象中的元素并释放内存空间
    elements = data.first; // 更新数据成员使其指向新空间
    first_free = cap = data.second;
    return *this;
}
```

和拷贝赋值及移动赋值运算符一样，其他重载的赋值运算符也必须先释放当前内存空间，再创建一片新空间。不同之处是，这个运算符无须检查对象向自身的赋值，这是因为它的形参 `initializer_list<string>`（参见 6.2.6 节，第 198 页）确保 `il` 与 `this` 所指的不是同一个对象。



我们可以重载赋值运算符。不论形参的类型是什么，赋值运算符都必须定义为成员函数。

复合赋值运算符

复合赋值运算符不非得是类的成员，不过我们还是倾向于把包括复合赋值在内的所有赋值运算都定义在类的内部。为了与内置类型的复合赋值保持一致，类中的复合赋值运算符也要返回其左侧运算对象的引用。例如，下面是 `Sales_data` 类中复合赋值运算符的定义：

```
// 作为成员的二元运算符：左侧运算对象绑定到隐式的 this 指针
// 假定两个对象表示的是同一本书
Sales_data& Sales_data::operator+=(const Sales_data &rhs)
{
    units_sold += rhs.units_sold;
    revenue += rhs.revenue;
    return *this;
}
```



赋值运算符必须定义成类的成员，复合赋值运算符通常情况下也应该这样做。这两类运算符都应该返回左侧运算对象的引用。

14.4 节练习

练习 14.20：为你的 `Sales_data` 类定义加法和复合赋值运算符。

练习 14.21：编写 `Sales_data` 类的 `+` 和 `+=` 运算符，使得 `+` 执行实际的加法操作而 `+=` 调用 `+`。相比于 14.3 节（第 497 页）和 14.4 节（第 500 页）对这两个运算符的定义，本题的定义有何缺点？试讨论之。

练习 14.22：定义赋值运算符的一个新版本，使得我们能把一个表示 ISBN 的 `string` 赋给一个 `Sales_data` 对象。

练习 14.23：为你的 `StrVec` 类定义一个 `initializer_list` 赋值运算符。

练习 14.24: 你在 7.5.1 节的练习 7.40 (第 261 页) 中曾经选择并编写了一个类, 你认为它应该含有拷贝赋值和移动赋值运算符吗? 如果是, 请实现它们。

练习 14.25: 上题的这个类还需要定义其他赋值运算符吗? 如果是, 请实现它们; 同时说明运算对象应该是什么类型并解释原因。

14.5 下标运算符

表示容器的类通常可以通过元素在容器中的位置访问元素, 这些类一般会定义下标运算符 `operator[]`。



下标运算符必须是成员函数。

565

为了与下标的原始定义兼容, 下标运算符通常以所访问元素的引用作为返回值, 这样做的好处是下标可以出现在赋值运算符的任意一端。进一步, 我们最好同时定义下标运算符的常量版本和非常量版本, 当作用于一个常量对象时, 下标运算符返回常量引用以确保我们不会给返回的对象赋值。



如果一个类包含下标运算符, 则它通常会定义两个版本: 一个返回普通引用, 另一个是类的常量成员并且返回常量引用。

举个例子, 我们按照如下形式定义 `StrVec` (参见 13.5 节, 第 465 页) 的下标运算符:

```
class StrVec {
public:
    std::string& operator[](std::size_t n)
        { return elements[n]; }
    const std::string& operator[](std::size_t n) const
        { return elements[n]; }
    // 其他成员与 13.5 (第 465 页) 一致
private:
    std::string *elements;           // 指向数组首元素的指针
};
```

上面这两个下标运算符的用法类似于 `vector` 或者数组中的下标。因为下标运算符返回的是元素的引用, 所以当 `StrVec` 是非常量时, 我们可以给元素赋值; 而当我们对常量对象取下标时, 不能为其赋值:

```
// 假设 svec 是一个 StrVec 对象
const StrVec cvec = svec;           // 把 svec 的元素拷贝到 cvec 中
// 如果 svec 中含有元素, 对第一个元素运行 string 的 empty 函数
if (svec.size() && svec[0].empty()) {
    svec[0] = "zero";               // 正确: 下标运算符返回 string 的引用
    cvec[0] = "Zip";                // 错误: 对 cvec 取下标返回的是常量引用
}
```

566